SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA1011	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 1 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	HARIHARAN.R	**Reg. No.:**	192524003
Date:	21.10.2026	**Duration:**	60 minutes

Code of Conduct

I ___HARIHARAN.R(192524003)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:HARIHARAN.R_____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

Requirements Validation for an Online Examination System: A software team has prepared a requirements specification for an online examination application. Analyze how the requirementscan be checked for correctness, completeness, consistency, feasibility, traceability, andtestability. Propose suitable validation techniques and sample acceptance criteria.

ANSWER::

Requirements Validation for an Online Examination System

1. Introduction

Requirements validation is the process of checking whether software requirements satisfy user needs. In an Online Examination System, it ensures that students can log in, attend examinations, submit answers, and view results correctly. The software team must validate the requirements before development to avoid errors and improve system quality.

2. Scenario Analysis

A software team has prepared a requirements specification for an Online Examination System. The team must check the requirements for correctness, completeness, consistency, feasibility, traceability, and testability. Suitable validation techniques and acceptance criteria are used to ensure that the system meets the needs of students, teachers, and administrators.

3. Requirements Validation Criteria

1. Correctness

Correctness ensures that the requirements match the actual needs of users.

Example: The system must save students' answers correctly when they submit an examination.

Validation Technique: Stakeholder interviews and requirements reviews.

2. Completeness
Completeness ensures that all necessary requirements are included in the specification.
Example: The system must include login, exam scheduling, question display, answer submission, evaluation, and result publication.
Validation Technique: Requirements checklists and use-case analysis.

3. Consistency
Consistency ensures that no two requirements contradict each other.
Example: One requirement must not allow unlimited submissions while another prohibits resubmission.
Validation Technique: Requirement reviews and consistency checking.

4. Feasibility
Feasibility checks whether the requirements can be implemented within the available budget, time, and technology.
Example: The system must be tested to confirm that it can support 5,000 students attending an examination simultaneously.
Validation Technique: Technical feasibility studies and performance testing.

5. Traceability
Traceability ensures that each requirement is linked to its source, design, implementation, and test case.
Example: The requirement for result generation must be linked to the result module and its corresponding test case.
Validation Technique: Requirements Traceability Matrix (RTM).

6. Testability
Testability ensures that requirements are measurable and can be verified through testing.
Example: The examination page must load within two seconds under specified test conditions.
Validation Technique: Test case preparation and measurable acceptance criteria.

4. Requirements Validation Data Box

Validation Criteria	Purpose	Example
Correctness	Meet user needs	Answers are saved correctly
Completeness	Include all features	Login, exam, submission, results
Consistency	Avoid conflicts	Clear submission rules
Feasibility	Check practicality	Support expected student load
Traceability	Track each requirement	Requirement linked to test case
Testability	Enable verification	Page loads within two seconds

5. Validation Techniques

Technique	Description	Application
Requirements Review	Inspect requirements for errors	Review exam rules
Prototyping	Create a sample interface	Collect student feedback
Test Case Generation	Prepare tests for requirements	Test answer submission

Use-Case Validation	Verify user activities	Check login and exam flow
Feasibility Analysis	Evaluate cost and technology	Check server capacity
Traceability Analysis	Map requirements to tests	Connect results feature to TC04

6. Sample Acceptance Criteria

Requirement	Acceptance Criteria	Test Result
Student Login	Valid credentials allow access	Pass/Fail
Exam Timer	Exam ends when the configured duration expires	Pass/Fail
Answer Submission	Submitted answers are saved correctly	Pass/Fail
Result Generation	Score matches evaluation rules	Pass/Fail
Performance	Page loads within two seconds under defined conditions	Pass/Fail
Security	Unauthorized users cannot access another student's results	Pass/Fail

7. Requirements Traceability Matrix

Requirement ID	Requirement	Validation Method	Test Case ID
R01	Student Login	Requirements Review	TC01
R02	Automatic Exam Timer	Functional Testing	TC02
R03	Answer Submission	Functional Testing	TC03

R04	Result Generation	Result Verification	TC04
R05	Access Control	Security Testing	TC05

8. Sample Test Case Data Box

Test Case	Input	Expected Output
TC01	Valid username and password	Login successful
TC02	Examination time expires	Exam closes automatically
TC03	Student submits answers	Answers saved successfully
TC04	Student completes exam	Correct score displayed
TC05	Student opens another student's results	Access denied

9. Requirement Validation Workflow::

Requirement Collection → Requirement Review → Consistency Checking → Feasibility Analysis → Acceptance Criteria Preparation → Test Case Mapping → Validation Approval.

10. Expected Outcome

1. Requirements correctly represent user needs.
2. Missing and conflicting requirements are identified.
3. Technical and budget limitations are understood.
4. Every requirement is linked to suitable test cases.
5. The system can be tested using measurable acceptance criteria.
6. Software quality, security, and reliability are improved.

11. Conclusion:

Requirements validation is an important step in developing an Online Examination System. By checking correctness, completeness, consistency, feasibility, traceability, and testability, the software team can identify problems before development. Requirements reviews, prototyping, feasibility analysis, traceability matrices, and acceptance criteria help ensure that the final system is accurate, secure, reliable, and user-friendly.